Elevator Term Project

Correctness By Construction

Christopher Roßbach David Hospodka Antonio José Fernández Pinto

Universidad Politécnica de Madrid

21 May 2025

Model Diagram

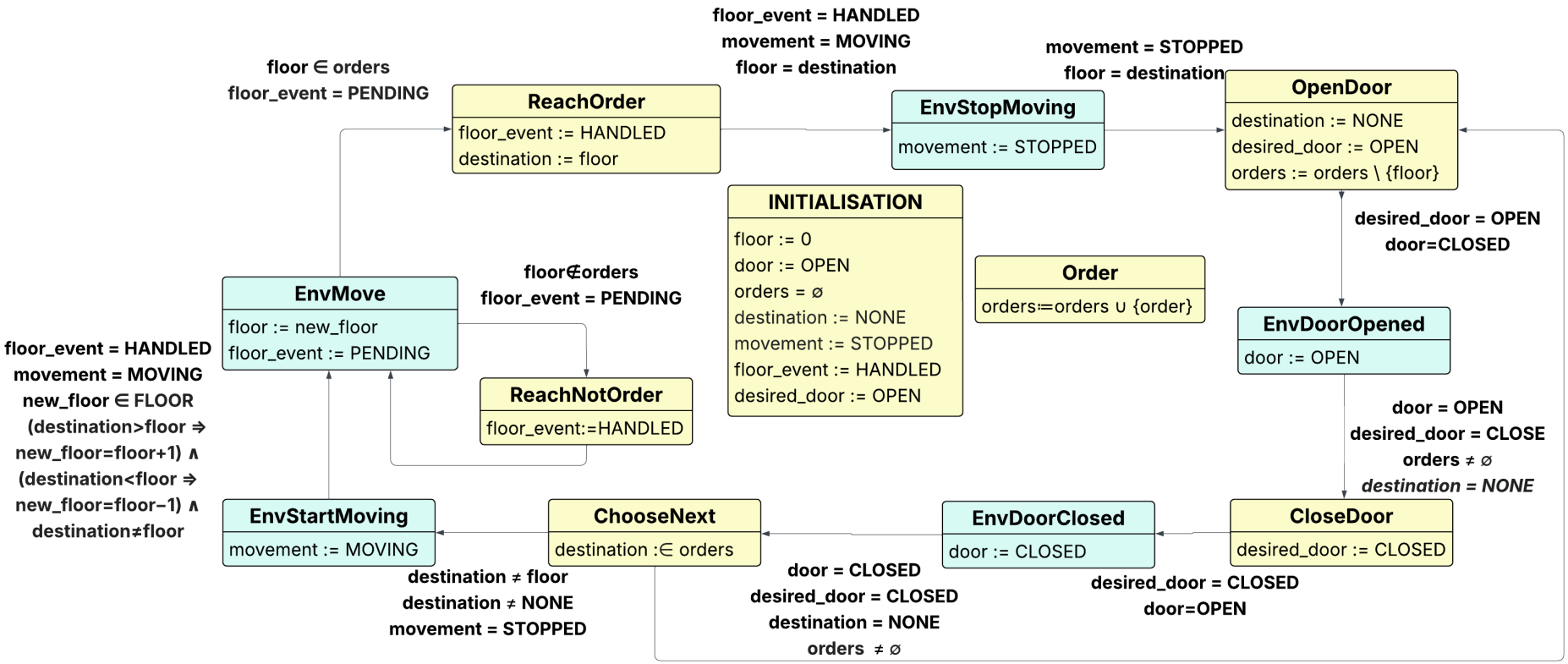

Figure: Diagram of the model

Invariants for Requirements: Elevator Movement

FUN 16	The elevator should not move in a direction where there are no pending requests to attend.

Invariants for Requirements: Elevator Movement

FUN 16	The elevator should not move in a direction where there are no pending requests to attend.

  inv_move_to_ordered: $\texttt{destination} \neq \texttt{NONE} \implies \texttt{destination} \in \texttt{orders}$

Invariants for Requirements: Elevator Movement

FUN 16	The elevator should not move in a direction where there are no pending requests to attend.

  $\texttt{inv\_move\_to\_ordered}: \texttt{destination} \neq \texttt{NONE} \implies \texttt{destination} \in \texttt{orders}$

FUN 15	The elevator should not move if there are no requests waiting to be served.

Invariants for Requirements: Elevator Movement

FUN 16	The elevator should not move in a direction where there are no pending requests to attend.

  $\texttt{inv\_move\_to\_ordered}:\texttt{destination} \neq \texttt{NONE} \implies \texttt{destination} \in \texttt{orders}$

FUN 15	The elevator should not move if there are no requests waiting to be served.

  $\texttt{inv\_only\_move\_with\_orders}^{\text{THM}}:\texttt{movement}=\texttt{MOVING} \implies \texttt{orders} \neq \emptyset$

  This follows directly from FUN 16 and the helper:

  $\texttt{inv\_only\_move\_when\_destination}:\texttt{movement}=\texttt{MOVING} \implies \texttt{destination} \neq \texttt{NONE}$

Invariants for Requirements: Door Management

- Necessary to ensure movement restrictions according to door status

Invariants for Requirements: Door Management

- Necessary to ensure movement restrictions according to door status

door	desired_door	Meaning
OPEN	OPEN	The door is open and stays open
OPEN	CLOSED	The door is closing
CLOSED	OPEN	The door is opening
CLOSED	CLOSED	The door is closed and stays closed

Invariants for Requirements: Door Management

- Necessary to ensure movement restrictions according to door status

door	desired_door	Meaning
OPEN	OPEN	The door is open and stays open
OPEN	CLOSED	The door is closing
CLOSED	OPEN	The door is opening
CLOSED	CLOSED	The door is closed and stays closed

- | FUN 8 | The elevator doors should be closed when the elevator is moving. |

Invariants for Requirements: Door Management

- Necessary to ensure movement restrictions according to door status

door	desired_door	Meaning
OPEN	OPEN	The door is open and stays open
OPEN	CLOSED	The door is closing
CLOSED	OPEN	The door is opening
CLOSED	CLOSED	The door is closed and stays closed

- | FUN 8 | The elevator doors should be closed when the elevator is moving. |

inv_door_closed_when_moving^{THM}:

movement=MOVING $\Longrightarrow$ door=CLOSED$\wedge$desired_door=CLOSED

Invariants for Requirements: Door Management

- Necessary to ensure movement restrictions according to door status

door	desired_door	Meaning
OPEN	OPEN	The door is open and stays open
OPEN	CLOSED	The door is closing
CLOSED	OPEN	The door is opening
CLOSED	CLOSED	The door is closed and stays closed

- | FUN 8 | The elevator doors should be closed when the elevator is moving. |

  $\texttt{inv\_door\_closed\_when\_moving}^{\text{THM}}$:

  $\texttt{movement=MOVING} \Longrightarrow \texttt{door=CLOSED} \wedge \texttt{desired\_door=CLOSED}$

  Helper:

  $\texttt{inv\_door\_closed\_when\_destination:destination} \neq \texttt{NONE} \Longrightarrow \texttt{door=CLOSED}$

  $\texttt{inv\_no\_dest\_when\_opening\_door:desired\_door=OPEN} \Longrightarrow \texttt{destination=NONE}$

Invariants for Requirements: Door Management

- Necessary to ensure movement restrictions according to door status

door	desired_door	Meaning
OPEN	OPEN	The door is open and stays open
OPEN	CLOSED	The door is closing
CLOSED	OPEN	The door is opening
CLOSED	CLOSED	The door is closed and stays closed

- | FUN 8 | The elevator doors should be closed when the elevator is moving. |

  $\texttt{inv\_door\_closed\_when\_moving}^{\text{THM}}$:

  $\texttt{movement=MOVING} \Longrightarrow \texttt{door=CLOSED} \wedge \texttt{desired\_door=CLOSED}$

  Helper:

  $\texttt{inv\_door\_closed\_when\_destination:destination} \neq \texttt{NONE} \Longrightarrow \texttt{door=CLOSED}$

  $\texttt{inv\_no\_dest\_when\_opening\_door:desired\_door=OPEN} \Longrightarrow \texttt{destination=NONE}$

- | FUN 11 | The doors should be open if there are no requests to serve. |

Invariants for Requirements: Door Management

- Necessary to ensure movement restrictions according to door status

door	desired_door	Meaning
OPEN	OPEN	The door is open and stays open
OPEN	CLOSED	The door is closing
CLOSED	OPEN	The door is opening
CLOSED	CLOSED	The door is closed and stays closed

- | FUN 8 | The elevator doors should be closed when the elevator is moving. |

  $\texttt{inv\_door\_closed\_when\_moving}^{\text{THM}}$:

  $\texttt{movement=MOVING} \implies \texttt{door=CLOSED} \wedge \texttt{desired\_door=CLOSED}$

  Helper:

  $\texttt{inv\_door\_closed\_when\_destination:destination} \neq \texttt{NONE} \implies \texttt{door=CLOSED}$

  $\texttt{inv\_no\_dest\_when\_opening\_door:desired\_door=OPEN} \implies \texttt{destination=NONE}$

- | FUN 11 | The doors should be open if there are no requests to serve. |

  $\texttt{inv\_no\_orders\_door\_open}^{\text{THM}}\texttt{:orders=}\emptyset \implies \texttt{door=OPEN} \vee \texttt{desired\_door=OPEN}$

Invariants for Requirements: Door Management

- Necessary to ensure movement restrictions according to door status

door	desired_door	Meaning
OPEN	OPEN	The door is open and stays open
OPEN	CLOSED	The door is closing
CLOSED	OPEN	The door is opening
CLOSED	CLOSED	The door is closed and stays closed

- | FUN 8 | The elevator doors should be closed when the elevator is moving. |

$\texttt{inv\_door\_closed\_when\_moving}^{\text{THM}}$:

$\texttt{movement=MOVING} \implies \texttt{door=CLOSED} \wedge \texttt{desired\_door=CLOSED}$

Helper:

$\texttt{inv\_door\_closed\_when\_destination:destination} \neq \texttt{NONE} \implies \texttt{door=CLOSED}$

$\texttt{inv\_no\_dest\_when\_opening\_door:desired\_door=OPEN} \implies \texttt{destination=NONE}$

- | FUN 11 | The doors should be open if there are no requests to serve. |

$\texttt{inv\_no\_orders\_door\_open}^{\text{THM}}\texttt{:orders=}\emptyset \implies \texttt{door=OPEN} \vee \texttt{desired\_door=OPEN}$

Helper:

$\texttt{inv\_orders\_when\_closing\_door:desired\_door=CLOSED} \implies \texttt{orders} \neq \emptyset$

The Need for Refinements: Deterministic Serving Order

- So far: **ChooseNext** selects `destination:∈orders` randomly

The Need for Refinements: Deterministic Serving Order

- So far: **ChooseNext** selects `destination:∈orders` randomly

FUN 13	All pending requests should be eventually be served.

The Need for Refinements: Deterministic Serving Order

- So far: **ChooseNext** selects `destination:∈orders` randomly

FUN 13	All pending requests should be eventually be served.

- Imagine this situation:

floor	orders	Description
0	{1,4}	We are at the ground floor. Floors 1 and 4 are ordered.
1	{4}	We select floor 1 and served it.
1	{0,4}	Somebody ordered the ground floor.
0	{4}	We select floor 0 and served it.
0	{1,4}	Somebody ordered floor 1. This can repeat infinitely.

The Algorithm

- We'll just use the elevator algorithm.

The Algorithm

- We'll just use the elevator algorithm.
- **`ChooseNext`** gets transformed into four new events.
  - **`ChangeDirectionUp`**
  - **`ChangeDirectionDown`**
  - **`ChooseNextUp`**
  - **`ChooseNextDown`**

Informal Argument for the Liveness of the Algorithm

- Properties this algorithm has:

Informal Argument for the Liveness of the Algorithm

- Properties this algorithm has:
  - The elevator moves/starts moving when there are any orders.

Informal Argument for the Liveness of the Algorithm

- Properties this algorithm has:
  - The elevator moves/starts moving when there are any orders.
  - The elevator stops at every floor that has been ordered and that is in its immediate path.

Informal Argument for the Liveness of the Algorithm

- Properties this algorithm has:
  - The elevator moves/starts moving when there are any orders.
  - The elevator stops at every floor that has been ordered and that is in its immediate path.
  - If the elevator is going up/down, then it continues to do so as long as there are orders above/below.

Informal Argument for the Liveness of the Algorithm

- Properties this algorithm has:
  - The elevator moves/starts moving when there are any orders.
  - The elevator stops at every floor that has been ordered and that is in its immediate path.
  - If the elevator is going up/down, then it continues to do so as long as there are orders above/below.
  - When it runs out of orders above/below, but it still has orders the other way, it changes direction.

Informal Argument for the Liveness of the Algorithm

- Properties this algorithm has:
  - The elevator moves/starts moving when there are any orders.
  - The elevator stops at every floor that has been ordered and that is in its immediate path.
  - If the elevator is going up/down, then it continues to do so as long as there are orders above/below.
  - When it runs out of orders above/below, but it still has orders the other way, it changes direction.
- Then, for any floor that is ordered:

Informal Argument for the Liveness of the Algorithm

- Properties this algorithm has:
  - The elevator moves/starts moving when there are any orders.
  - The elevator stops at every floor that has been ordered and that is in its immediate path.
  - If the elevator is going up/down, then it continues to do so as long as there are orders above/below.
  - When it runs out of orders above/below, but it still has orders the other way, it changes direction.
- Then, for any floor that is ordered:
  - The elevator may be at that floor already, so it is served.

Informal Argument for the Liveness of the Algorithm

- Properties this algorithm has:
  - The elevator moves/starts moving when there are any orders.
  - The elevator stops at every floor that has been ordered and that is in its immediate path.
  - If the elevator is going up/down, then it continues to do so as long as there are orders above/below.
  - When it runs out of orders above/below, but it still has orders the other way, it changes direction.
- Then, for any floor that is ordered:
  - The elevator may be at that floor already, so it is served.
  - The elevator is not currently at that floor. Then either:

Informal Argument for the Liveness of the Algorithm

- Properties this algorithm has:
  - The elevator moves/starts moving when there are any orders.
  - The elevator stops at every floor that has been ordered and that is in its immediate path.
  - If the elevator is going up/down, then it continues to do so as long as there are orders above/below.
  - When it runs out of orders above/below, but it still has orders the other way, it changes direction.
- Then, for any floor that is ordered:
  - The elevator may be at that floor already, so it is served.
  - The elevator is not currently at that floor. Then either:
    - ⋆ It's moving in the opposite direction, in which case it will eventually turn around.

Informal Argument for the Liveness of the Algorithm

- Properties this algorithm has:
  - The elevator moves/starts moving when there are any orders.
  - The elevator stops at every floor that has been ordered and that is in its immediate path.
  - If the elevator is going up/down, then it continues to do so as long as there are orders above/below.
  - When it runs out of orders above/below, but it still has orders the other way, it changes direction.
- Then, for any floor that is ordered:
  - The elevator may be at that floor already, so it is served.
  - The elevator is not currently at that floor. Then either:
    - ⋆ It's moving in the opposite direction, in which case it will eventually turn around.
    - ⋆ It's moving in the correct direction, in which case it will eventually reach the ordered floor and serve it.

Refinements Diagram

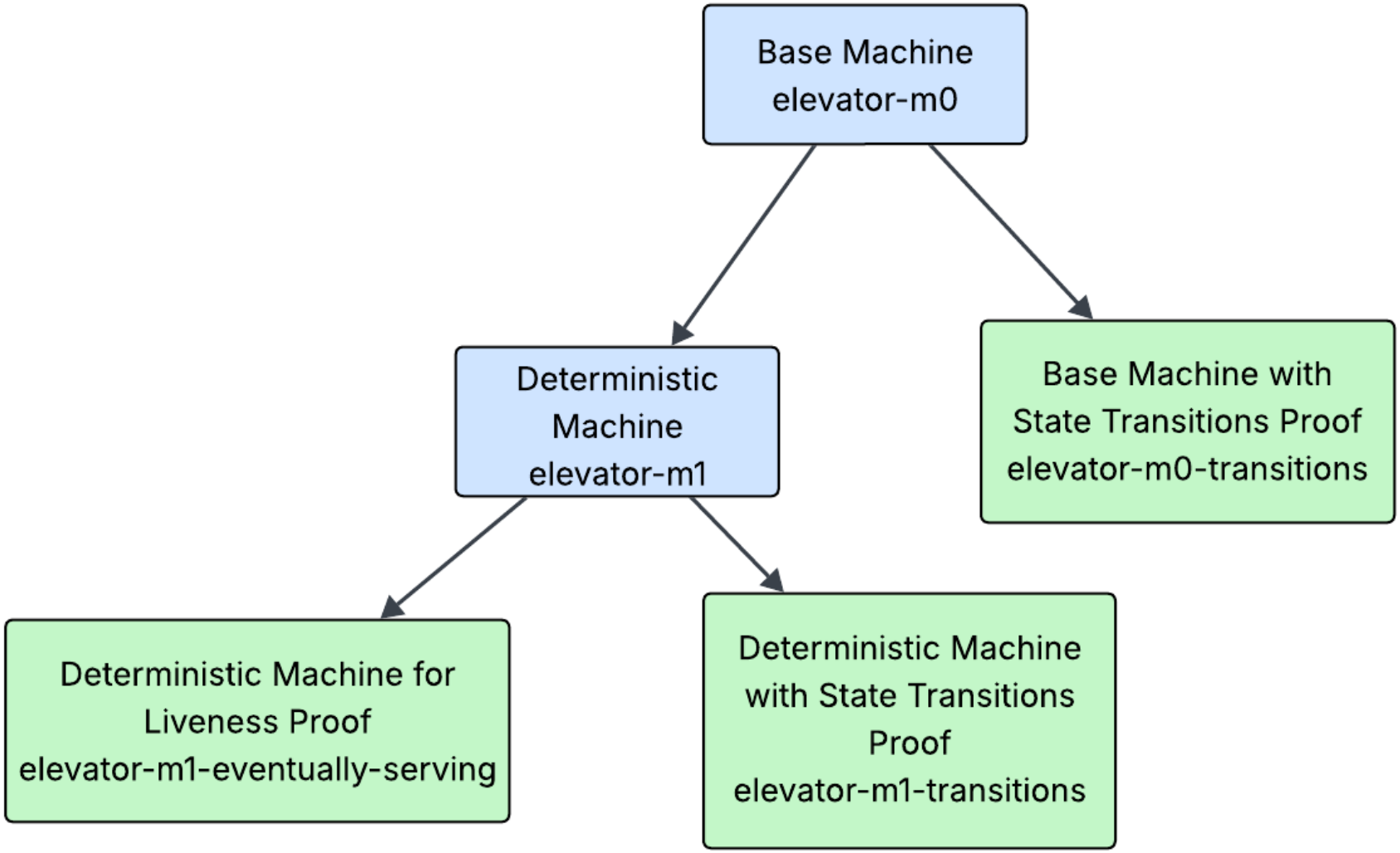

Figure: Refinements family tree

Realization in Rodin

- Add a refinement to isolate everything cleanly

Realization in Rodin

- Add a refinement to isolate everything cleanly
- Define a variable `waiting_since` that tracks the waiting time for each order: `waiting_type:waiting_since`$\in$`FLOOR`$\rightarrow \mathbb{N}$

Realization in Rodin

- Add a refinement to isolate everything cleanly
- Define a variable `waiting_since` that tracks the waiting time for each order: `waiting_type:waiting_since`$\in$`FLOOR`$\rightarrow \mathbb{N}$
- Assume an upper limit on the waiting time using an invariant `waiting_limit:`$\forall x \cdot x \in$`FLOOR`$\implies$`waiting_since`(x) $<$`N_Levels`$*2$

Realization in Rodin

- Add a refinement to isolate everything cleanly
- Define a variable `waiting_since` that tracks the waiting time for each order:
  `waiting_type:waiting_since`$\in$`FLOOR`$\rightarrow \mathbb{N}$
- Assume an upper limit on the waiting time using an invariant
  `waiting_limit:`$\forall x \cdot x \in$`FLOOR`$\implies$`waiting_since`(x) $<$`N_Levels*2`
- In **`OpenDoor`** add the Action
  `act_inc_and_reset:waiting_since:=`
  `((waiting_since;add_one)`$\lhd\!+$`{floor`$\mapsto$`0})`$\lhd\!+$`((FLOOR`$\setminus$`orders)`$\times${`0})`

Realization in Rodin

- Add a refinement to isolate everything cleanly
- Define a variable `waiting_since` that tracks the waiting time for each order:
  `waiting_type:waiting_since∈FLOOR→ ℕ`
- Assume an upper limit on the waiting time using an invariant
  `waiting_limit:∀x · x ∈FLOOR ⟹ waiting_since(x) <N_Levels∗2`
- In **`OpenDoor`** add the Action
  `act_inc_and_reset:waiting_since:=`
  `((waiting_since;add_one)⩤{floor↦0})⩤((FLOOR∖orders)×{0})`
- Where `add_one` is defined as a constant in the context with the axioms
  - `add_one∈ ℕ → ℕ1`
  - `∀n·n∈ ℕ ⟹ add_one(n)=n+1`

Realization in Rodin

- Add a refinement to isolate everything cleanly
- Define a variable `waiting_since` that tracks the waiting time for each order:
  `waiting_type:waiting_since∈FLOOR→ ℕ`
- Assume an upper limit on the waiting time using an invariant
  `waiting_limit:∀x · x ∈FLOOR ⟹ waiting_since(x) <N_Levels∗2`
- In **`OpenDoor`** add the Action
  `act_inc_and_reset:waiting_since:=`
  `((waiting_since;add_one)⩤{floor↦0})⩤((FLOOR∖orders)×{0})`
- Where `add_one` is defined as a constant in the context with the axioms
  - `add_one∈ ℕ → ℕ1`
  - `∀n·n∈ ℕ ⟹ add_one(n)=n+1`
- We were not able to proof this invariant in Rodin

Realization in Rodin

- Add a refinement to isolate everything cleanly
- Define a variable `waiting_since` that tracks the waiting time for each order:
  `waiting_type:waiting_since∈FLOOR→ ℕ`
- Assume an upper limit on the waiting time using an invariant
  `waiting_limit:`$\forall x \cdot x \in$`FLOOR` $\implies$ `waiting_since`(x) `<N_Levels*2`
- In **`OpenDoor`** add the Action
  `act_inc_and_reset:waiting_since:=`
  `((waiting_since;add_one)<+{floor↦0})<+((FLOOR\orders)×{0})`
- Where `add_one` is defined as a constant in the context with the axioms
  - `add_one∈ ℕ → ℕ1`
  - `∀n·n∈ ℕ` $\implies$ `add_one(n)=n+1`
- We were not able to proof this invariant in Rodin
- We did ProB model checking to search for invariant violations

Realization in Rodin

- Add a refinement to isolate everything cleanly
- Define a variable `waiting_since` that tracks the waiting time for each order:
  `waiting_type:waiting_since∈FLOOR→ ℕ`
- Assume an upper limit on the waiting time using an invariant
  `waiting_limit:`$\forall x \cdot x \in$`FLOOR` $\implies$ `waiting_since(`x`) <N_Levels*2`
- In **`OpenDoor`** add the Action
  `act_inc_and_reset:waiting_since:=`
  `((waiting_since;add_one)<+{floor↦0})<+((FLOOR\orders)×{0})`
- Where `add_one` is defined as a constant in the context with the axioms
  - `add_one∈ ℕ → ℕ1`
  - `∀n·n∈ ℕ` $\implies$ `add_one(n)=n+1`
- We were not able to proof this invariant in Rodin
- We did ProB model checking to search for invariant violations
  - Check with different numbers of floors `N_Levels∈{1,2,3,4,5}`

Realization in Rodin

- Add a refinement to isolate everything cleanly
- Define a variable `waiting_since` that tracks the waiting time for each order:
  `waiting_type:waiting_since∈FLOOR→ ℕ`
- Assume an upper limit on the waiting time using an invariant
  `waiting_limit:`$\forall x \cdot x \in$`FLOOR` $\implies$ `waiting_since(`x`) <N_Levels*2`
- In **`OpenDoor`** add the Action
  `act_inc_and_reset:waiting_since:=`
  `((waiting_since;add_one)<+{floor↦0})<+((FLOOR\orders)×{0})`
- Where `add_one` is defined as a constant in the context with the axioms
  - `add_one∈ ℕ → ℕ1`
  - `∀n·n∈ ℕ` $\implies$ `add_one(n)=n+1`
- We were not able to proof this invariant in Rodin
- We did ProB model checking to search for invariant violations
  - Check with different numbers of floors `N_Levels∈{1,2,3,4,5}`
  - No invariant violations found

Realization in Rodin

- Add a refinement to isolate everything cleanly
- Define a variable `waiting_since` that tracks the waiting time for each order:
  `waiting_type:waiting_since∈FLOOR→ ℕ`
- Assume an upper limit on the waiting time using an invariant
  `waiting_limit:`$\forall x \cdot x \in$`FLOOR` $\implies$ `waiting_since(`x`) <N_Levels*2`
- In **`OpenDoor`** add the Action
  `act_inc_and_reset:waiting_since:=`
  `((waiting_since;add_one)<+{floor↦0})<+((FLOOR\orders)×{0})`
- Where `add_one` is defined as a constant in the context with the axioms
  - `add_one∈ ℕ → ℕ1`
  - `∀n·n∈ ℕ` $\implies$ `add_one(n)=n+1`
- We were not able to proof this invariant in Rodin
- We did ProB model checking to search for invariant violations
  - Check with different numbers of floors `N_Levels∈{1,2,3,4,5}`
  - No invariant violations found
  - Mark as reviewed

Realization in Rodin

- Add a refinement to isolate everything cleanly
- Define a variable `waiting_since` that tracks the waiting time for each order:
  `waiting_type:waiting_since∈FLOOR→ ℕ`
- Assume an upper limit on the waiting time using an invariant
  `waiting_limit:`$\forall x \cdot x \in$`FLOOR` $\implies$ `waiting_since(`x`) <N_Levels*2`
- In **`OpenDoor`** add the Action
  `act_inc_and_reset:waiting_since:=`
  `((waiting_since;add_one)<+{floor↦0})<+((FLOOR\orders)×{0})`
- Where `add_one` is defined as a constant in the context with the axioms
  - `add_one∈ ℕ → ℕ1`
  - `∀n·n∈ ℕ` $\implies$ `add_one(n)=n+1`
- We were not able to proof this invariant in Rodin
- We did ProB model checking to search for invariant violations
  - Check with different numbers of floors `N_Levels∈{1,2,3,4,5}`
  - No invariant violations found
  - Mark as reviewed
  - Workaround for ProB `add_one:add_one:={0↦1, 1↦2, 2↦3, ...}`

Deadlock Freedom

- When there are orders, we want to ensure there is always a possible event, that is not **`Order`**.

Deadlock Freedom

- When there are orders, we want to ensure there is always a possible event, that is not **Order**.
- $\texttt{inv\_no\_deadlock}^{\text{THM}}\texttt{:orders} \neq \varnothing \implies (\texttt{guards for } \mathbf{\texttt{EnvMove}}) \vee (\texttt{guards for } \mathbf{\texttt{ReachOrder}}) \vee (\texttt{guards for } \mathbf{\texttt{ReachNotOrder}}) \vee \ldots$

State Transitions

- The guards for the events are fairly complicated, so it's not obvious they follow each other like the arrows of the shown diagrams. For instance, does **`EnvStopMoving`** always come after, and only after, **`ReachOrder`**.

State Transitions

- The guards for the events are fairly complicated, so it's not obvious they follow each other like the arrows of the shown diagrams. For instance, does **`EnvStopMoving`** always come after, and only after, **`ReachOrder`**.
- To prove this, we can make a refinement that doesn't change the behaviour of the base model, but adds a `state` variable that is set to a different value for every possible state in between events.

State Transitions

- The guards for the events are fairly complicated, so it's not obvious they follow each other like the arrows of the shown diagrams. For instance, does **`EnvStopMoving`** always come after, and only after, **`ReachOrder`**.
- To prove this, we can make a refinement that doesn't change the behaviour of the base model, but adds a `state` variable that is set to a different value for every possible state in between events.
- Only **`ReachOrder`** sets `state` to 6, and `inv6:state` $= 6 \Leftrightarrow$ (`guards for` **`EnvStopMoving`**).

Thank you for your attention!

Reading from the Environment

- The controller is able to read the environment state directly

Reading from the Environment

- The controller is able to read the environment state directly

Variable	Value	Description
`movement`	`MOVING`	The elevator is (physically) moving
	`STOPPED`	The elevator is (physically) not moving
`door`	`OPEN`	The elevator door is (physically) open†
	`CLOSED`	The elevator door is (physically) closed†
`floor`	`0..N_Levels`	The floor the elevator is (physically) on‡

† the value changes as soon as the door is fully open/closed

‡ the next possible floor to stop

Reading from the Environment

- The controller is able to read the environment state directly

Variable	Value	Description
`movement`	`MOVING`	The elevator is (physically) moving
	`STOPPED`	The elevator is (physically) not moving
`door`	`OPEN`	The elevator door is (physically) open†
	`CLOSED`	The elevator door is (physically) closed†
`floor`	`0..N_Levels`	The floor the elevator is (physically) on‡

† the value changes as soon as the door is fully open/closed

‡ the next possible floor to stop

- The controller can read and set the order button state

Reading from the Environment

- The controller is able to read the environment state directly

Variable	Value	Description
`movement`	`MOVING`	The elevator is (physically) moving
	`STOPPED`	The elevator is (physically) not moving
`door`	`OPEN`	The elevator door is (physically) open†
	`CLOSED`	The elevator door is (physically) closed†
`floor`	`0..N_Levels`	The floor the elevator is (physically) on‡

† the value changes as soon as the door is fully open/closed

‡ the next possible floor to stop

- The controller can read and set the order button state
  - We consider changes to this variable to be instantaneous

Reading from the Environment

- The controller is able to read the environment state directly

Variable	Value	Description
`movement`	`MOVING`	The elevator is (physically) moving
	`STOPPED`	The elevator is (physically) not moving
`door`	`OPEN`	The elevator door is (physically) open†
	`CLOSED`	The elevator door is (physically) closed†
`floor`	`0..N_Levels`	The floor the elevator is (physically) on‡

† the value changes as soon as the door is fully open/closed

‡ the next possible floor to stop

- The controller can read and set the order button state
  - We consider changes to this variable to be instantaneous

FUN 12	The status of the buttons can be accessed from the controller.

Reading from the Environment

- The controller is able to read the environment state directly

Variable	Value	Description
`movement`	`MOVING`	The elevator is (physically) moving
	`STOPPED`	The elevator is (physically) not moving
`door`	`OPEN`	The elevator door is (physically) open†
	`CLOSED`	The elevator door is (physically) closed†
`floor`	`0..N_Levels`	The floor the elevator is (physically) on‡

† the value changes as soon as the door is fully open/closed

‡ the next possible floor to stop

- The controller can read and set the order button state
  - We consider changes to this variable to be instantaneous

FUN 12	The status of the buttons can be accessed from the controller.

Variable	Value	Description
`orders`	$\subseteq$ `floor`	The set of buttons where the state is `ON`

Controlling the Environment

- The controller can communicate with the environment by giving commands

Controlling the Environment

- The controller can communicate with the environment by giving commands

FUN 6	The elevator can receive a signal to move to any level and will stop automatically when it is reached [...].

Controlling the Environment

- The controller can communicate with the environment by giving commands

FUN 6	The elevator can receive a signal to move to any level and will stop automatically when it is reached [...].

FUN 7	The elevator has doors that can be closed or opened from a controller.

Controlling the Environment

- The controller can communicate with the environment by giving commands

FUN 6	The elevator can receive a signal to move to any level and will stop automatically when it is reached [...].

FUN 7	The elevator has doors that can be closed or opened from a controller.

Variable	Value	Description
destination	$\in$0..N_Levels	The elevator shall move to floor
	NONE	The elevator shall not move
desired_door	OPEN	The elevator shall open the door
	CLOSED	The elevator shall close the door

Controlling the Environment

- The controller can communicate with the environment by giving commands

FUN 6	The elevator can receive a signal to move to any level and will stop automatically when it is reached [...].

FUN 7	The elevator has doors that can be closed or opened from a controller.

Variable	Value	Description
`destination`	$\in$`0..N_Levels`	The elevator shall move to floor
	`NONE`	The elevator shall not move
`desired_door`	`OPEN`	The elevator shall open the door
	`CLOSED`	The elevator shall close the door

- Stopping $\Longrightarrow$ this is handled by `destination=floor`

Possible Improvements: Door Modelling

- The modelling of the physical door state is not completely "clean"
  - E.g. the `door` can be `CLOSED` while the door is currently opening
- Options to fix this came to our minds:
  1. Add additional physical states `OPENING` and `CLOSING`
     - ★ New environment events **`EnvStartOpening`** and **`StartClosing`** would be needed to adapt the physical state when `door≠desired_door`
     - ★ This adds complexity to the model
     - ★ It contradicts the idea, that `desired_door=OPEN` instantly triggers the opening
  2. Add a function `physical_door_state∈(DOOR×DOOR)→{OPEN, OPENING, CLOSED, CLOSING}`, that interprets `door` and `desired_door`
     - ★ Usage would change for example the invariant `inv_no_orders_door_open` which was `orders=∅ ⟹ door=OPEN∨desired_door=OPEN`
       to `orders=∅ ⟹ (physical_door_state(door, desired_door)∈{OPEN, OPENING})`
     - ★ This can improve readability
     - ★ But in increases proof complexity
  3. Use a single variable for `door` which is one of `OPEN, OPENING, CLOSED, CLOSING`
     - ★ Setting `door:=CLOSING` by the controller would make the door start closing
     - ★ The environment would set this variable upon finishing to `door:=CLOSED`
     - ★ This contradicts the idea, that the variables are only changed by the either environment or the controller
     - ★ This exception may be acceptable since `orders` behaves in a similar way

Reduce Direction Events

- We can define a helper function cut(floor, orders, direction) that returns the remaining orders in the direction of the current movement
- This functions has the axioms:
  - axm_cut_type: $cut \in (FLOOR \times \mathbb{P}(FLOOR) \times DIRECTION) \implies \mathbb{P}(FLOOR)$
  - axm_cut_up:
    $\forall floor, orders, direction \cdot$
    $floor \in FLOOR \land orders \subseteq FLOOR \land direction \in DIRECTION \land direction = UP$
    $\implies cut(floor \mapsto orders \mapsto direction) = \{x \mid x \in orders \land x > floor\}$
  - axm_cut_down:
    $\forall floor, orders, direction \cdot$
    $floor \in FLOOR \land orders \subseteq FLOOR \land direction \in DIRECTION \land direction = DOWN$
    $\implies cut(floor \mapsto orders \mapsto direction) = \{x \mid x \in orders \land x < floor\}$

Reduce Direction Events

- This would lead to reduction in events and avoids copy-paste-errors:
  - events **ChangeDirectionUp** and **ChangeDirectionDown** can be unified
    - ★ Common guards and actions stay
    - ★ The guard `cut(floor↦orders↦direction)=∅`
    - ★ and the action `direction:∈DIRECTION\{direction}` are added
  - events **ChooseNextUp** and **ChooseNextDown** can be unified
    - ★ Common guards and actions stay
    - ★ The guard `cut(floor↦orders↦direction)≠ ∅`
    - ★ and the action `direction:∈cut(floor↦orders↦direction)`
    - ★ To choose deterministic `max` or `min` could be used but this is not necessary
    - ★ Even a function like `next_cut` that chooses the `max` or `min` of the `cut`-set depending on the `direction` is possible
- We implemented this but a lot of proofs required repeated manual intervention as we made changes to the model
- Using the separate events made the proof obligations discharge automatically which was a big help in the iterative development, so we chose the separate event approach